

AXI VDMA sincronización PL-PS

1. Objetivo y contexto

OBJ1: analizar el IP AXIVDMA para optimizar la sincronización entre la captura de la imagen y su procesamiento por el software

En la versión base de la arquitectura Sw-Acel, la forma de funcionamiento es la siguiente:

- La cámara envía frames de forma continua (60fps, 16,6ms/f) mientras que, inicialmente, el VDMA está bloqueado y los pixels no se escriben en la DDR.
- Cuando el sw necesita un nuevo frame desbloquea el VDMA para que empiece a transmitir una nueva imagen. VDMA debe esperar a que finalice el frame en curso y comience un nuevo frame (TUSER) para escribirlo en la DDR. En el peor de los casos esto supone 33,2ms (16,6ms del frame en curso más 16,6ms del nuevo frame), y en el mejor de los casos 16,6ms.
- Una vez finalizada la escritura del frame, se notifica al sw a través de un flag y el VDMA se vuelve a bloquear.
- A partir de aquí el Sw pipeline inicia el procesamiento

El tiempo de procesamiento de un frame es $t = t_{cap} + t_{proc}$, siendo $t_{proc} \gg t_{cap}$. Esto supone que se pierden frames entre dos procesamientos consecutivos (frame drop). El objetivo es formar un pipeline entre la etapa de captura y la de procesamiento, de forma que $t = \max\{t_{proc}, t_{cap}\}$.

La solución habitual es utilizar un triple-buffer, de forma que el productor de imágenes (la cámara) va llenando los buffers con frames consecutivos y el consumidor (pipeline software) siempre dispone de un frame nuevo accediendo al último buffer escrito por el productor. En nuestro caso $t_{proc} \gg t_{cap}$, y además t_{proc} es variable (p.ej. con el número de keypoints a procesar)

Productor	Consumidor
Wr_b1_f1	-
Wr_b2_f2	Rd_b1_f1
Wr_b3_f3	
Wr_b2_f4	
Wr_b3_f5	
Wr_b2_f6	
Wr_b3_f7	Rd_b2_f6
Wr_b1_f8	
Wr_b3_f9	
Wr_b1_f10	
Wr_b3_f11	Rd_b1_f10
Wr_b1_f12	
Wr_b3_f13	

2. AXI VDMA

Soporte para productor/consumidor a distintos framerate, casos de uso:

a) Sincronización del canal de escritura y lectura del mismo VDMA. Es la utilización típica, p. ej: captura de frame (cámara, tpf, etc..) por VDMA y lectura VDMA para visualización (display port, hdmi, etc.). Diseño de referencia triple buffer:

https://support.xilinx.com/s/article/938327?language=en_US

Problema: en todos los casos el productor y el consumidor es un canal DMA, no existe el caso para productor DMA y consumidor PS.

b) Sincronización entre varios VDMA. Diseño de referencia:

c) Ping-pong buffer y buffer circular para casos específicos donde los tiempos del productor y el consumidor son constantes y se conocen de antemano. Diseño de referencia: <https://arcoresearch.com/2018/03/23/realizing-the-lucas-kanade-motion-estimation-algorithm-on-xilinx-zc702-board-for-full-hd-real-time-video-analysis/>

d) Sincronización captura/procesamiento (sw)

https://support.xilinx.com/s/question/0D52E00006hpfyASAQ/video-processing-on-zynq-baremetal-vdma-trouble?language=en_US

Sincronización Genlock:

- Genlock static, el slave observa donde está trabajando el master (frame_ptr) y evita el buffer del máster. En nuestro caso deberíamos trabajar como WriteSlave/ReadMaster el canal de escritura escribe todos los frames y evita el buffer del canal de lectura mediante un stride (configurable en regs). El canal slave tiene una entrada frame_ptr_in (indica el puntero donde está trabajando el master)
- Genlock dynamic, ambos slave y master conocen donde está trabajando cada uno (fram_ptr)

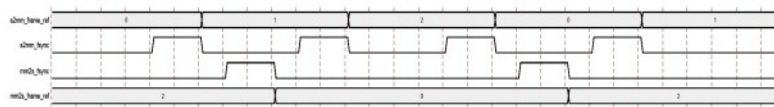
Ejemplos:

1) Genlock operation.

- Write(S2MM) Master / Read(MM2S) Slave Write
- frame rate Write >> frame rate Read

In the time the Write channel cycles through frame 0, 1, 2 and back to 0, the Read channel has only cycled through two frame. Due to the slow frame rate of the Read channel compared to the Write channel, the Read channel processes frame 2 then frame 0 then frame 2 again, skipping frame 1.

X-Ref Target - Figure 2-23



The Genlock Master uses the index of the Start Address register to specify which Start Address register the Genlock Slave should use. This Start Address register index is encoded as a Gray code value and appears on mm2s_frame_ptr_out and s2mm_frame_ptr_out for the MM2S and S2MM channels respectively.

2) Genlock dynamic operation. (NUESTRO CASO)

- Write(S2MM) Master / Read(MM2S) Slave Write
- frame rate Write >> frame rate Read

In Dynamic Genlock operation

- Master does not step onto the Slave current working frame.
- Slave works on the last completed frame by Master.

In [This Figure](#), after cycling through frame 0, 1, 2, the Write channel finds the Read channel on frame 0, and therefore skips this frame and moves to frame 1. The Read channel processes the last completed frame by the Write channel.

3. Propuestas de solución

1) triple buffer y Genlock dynamic

- VDMA write channel (S2MM) es el Master. Comunica el número de frame que fue escrito y lee el número de frame en el que está trabajando el Slave (sw).
- Sw es el Slave. Comunica el número del frame en el que está trabajando y lee el número de frame en el que escribió el Master.

Registros implicados:

- S2MM_VDMACR (S2MM VDMA Control Register - **Offset 30h**) Control Register:
 - * GenlockEn: enables Dynamic Genlock

- * Circular_Park: frame buffer ciclar (se alternan los buffers) o park (siempre el mismo)
 - S2MM_VDMASR (S2MM VDMA Status Register – **Offset 34h**) Status Register:
 - * IRQFrameCntSts: This field indicates the current value of frame count
 - S2MM Start Addresses (Offsets 0xAC to Maximum Offset 0xE8): Indicates the Start Address for video buffer 1 to N
 - PARK_PTR_REG (Park Pointer Register – Offset 28h)
 - * WrFrmStore: Write Frame Store number. Indicates current frame number being operated on by the S2MM channel. During VDMA operations this value continually updates as each frame is processed.
- Advanced options:
- **FRMPTR_STS** (MM2S and S2MM Current Frame Pointer Status – **Offset 24h**): This register provides the current operating frame pointer status of both MM2S and S2MM channels
 - * S2MMFrmPtrIn: Indicates Dynamic Genlock Slave frame pointer value if S2MM channel is Dynamic Genlock Master
 - * S2MMFrmPtrOut: Indicates current working frame of S2MM channel.

Proyecto: ImgCnd

- Configuración del IP VDMA:

- Registros

a) configuración inicial

```
videomode = VideoMode(1280, 720, 24)
mipi.configure(videomode)
----- VDMA Config -----
Size= 3840 x 720
stride= 3840
Park_ptr= 0x100
- WrFrmStore= 0
- WrFrmPtrRef= 1
S2MM_ControlReg= 0x11000
- GenlockEn= 0 Genlock disabled
- Circ_park= 0
S2MM_StatusReg= 0x11001
- IRQFrmCntSts= 1
S2MM_addr1= 0x0
S2MM_addr2= 0x0
S2MM_addr3= 0x0
S2MM_addr4= 0x0
S2MM_addr5= 0x0
FRMPTR_sts= 0
```

b) configuración para captura de imagen

```
frame = mipi.readframe()
Park_ptr= 0x2000100
- WrFrmStore= 2 Cambia cada vez que leemos frame
- WrFrmPtrRef= 1 Constante
S2MM_ControlReg= 0x11003 Constante
- GenlockEn= 0 Genlock disabled
- Circ_park= 1 Circular Mode
S2MM_StatusReg= 0x10000 Constante
- IRQFrmCntSts= 1
S2MM_addr1= 0x37d00000
S2MM_addr2= 0x37100000
S2MM_addr3= 0x37400000
```

```
S2MM_addr4= 0x37a00000
FRMPTR_sts= 0 Constante
```

c) análisis de `mipi.readframe()`

[https://github.com/Xilinx/PYNQ/tree/](https://github.com/Xilinx/PYNQ/tree/a056b8455f80a145839177102288e1f1d2b8ebe3/pynq/lib)

[a056b8455f80a145839177102288e1f1d2b8ebe3/pynq/lib](https://github.com/Xilinx/PYNQ/tree/a056b8455f80a145839177102288e1f1d2b8ebe3/pynq/lib)

`pcam5c.py`

```
def readframe(self):
    """Read a video frame
    See AxiVDMA.S2MMChannel.readframe for details
    """
    return self._vdma.readchannel.readframe()
```

Demasiado complejo para abordar su modificación

4. Búsqueda en Xilinx Forum

- **Reading frame and advance in a streaming video (using a VDMA)**

https://support.xilinx.com/s/question/0D52E00006hpdhmSAA/reading-frame-and-advance-in-a-streaming-video-using-a-vdma?language=en_US

```
dispCtrl.vdmaConfig.EnableCircularBuf = 0; // 0 = Park Mode; 1 = Circular Mode
dispCtrl.vdmaConfig.EnableFrameCounter = 0;//1; // 0 = stream forever; 1 = N frames
(def.1)
dispCtrl.vdmaConfig.FrameStoreStartAddr[0] = (UINTPTR)(fbs \+ 0); // frame addresses
dispCtrl.vdmaConfig.FixedFrameStoreAddr = 0; // Frame to use when in Park Mode
// Frame synchronization
dispCtrl.vdmaConfig.FrameDelay = 0; // min # of FBs Genlock slave is behind master
dispCtrl.vdmaConfig.EnableSync = 0; // GenLock sync (0 = disable; 1 = enable)
dispCtrl.vdmaConfig.PointNum = 0; // master in control when not GenLock Master mode
dispCtrl.vdmaConfig.GenLockRepeat = 0; // on error, 0 = drop frame; 1 = repeat frame
```

- **Video processing in PS using VDMA**

https://support.xilinx.com/s/question/0D52E00006hpWSFSA2/video-processing-in-ps-using-vdma?language=en_US

1- When a S2MM IRQ is got which register should be read to learn the frame buffer which written last from VDMA.

`PARK_PTR_REG[24:28] = WrFrmStore ?`

[Florent] - Well in your case, you should probably control everything from the processor. So your application should keep track of it

2- When a S2MM IRQ is got, of course i can learn last frame buffer number and i can apply algorithms on the data in PS, but what is the best time to write the data again on DDR? (I mean e.g. blurred image data here) Note: Why i write again? Because i want to display blurred images.

[Florent]- It depends on how many frame buffer you have. With 6 frame buffers you can do everything in parallel. Basically you do a triple frame buffer use case between the VDMA write and PS read and then a triple frame buffer between the PS write and VDMA read.

3- If i write the blurred data when operation done, it can coincide the reading time by MM2S channel. When to refresh DDR with new data? I don't want to disrupt the well working frame buffer mechanism VDMA itself?

[Florent]- Refer to my previous answer. Same, you can avoid this by using 6 frame buffers

4- What is the meaning of Frame delay in MM2S `FRMDELAY_STRIDE[24:28]` Register? For a camera->screen passthrough application, current frame on the screen, in fact before 4 frames ago camera data in case 4 frames delay? This means we have more time for processing? Is it true?

Florent] - This is not exactly true. Because if your PS is not as fast as your input you will still lose frame. It can give you some latency if you want to work on multiple frames at the same time

- **VDMA Dynamic Master Genlock with external pointer**

Hi, I am using the VDMA IP Core to transfer frames from a camera to DDR RAM. I want to process the last captured image present in the RAM, however I don't want it to get overwritten during processing. Meanwhile I want to keep fetching images from the camera so that when I'm done with one, I can move on to the last one acquired. Since I want to continuously fetch images I'm using the circular mode and that's working fine. However I can't figure out how to lock the address of the image I'm currently processing. According to the datasheet, Dynamic Master Genlock should skip the address indicated by s2mm_frame_ptr_in but it is not working for me. The buffer keeps iterating among buffers 0, 1 and 2. I have s2mm_frame_ptr_in set to 0, yet somehow it is still being overwritten.

I have mm2s disabled, Genlock is enabled and GenSource is 0 (since I want to control s2mm_frame_ptr_in externally). What am I doing wrong? Maybe Dynamic Master Genlock only works when frame_ptr is controlled internally?

[A] Your use case sounds fine and it's exactly a motivation for the external genlock feature. You have it set up for a triple buffer, it sounds like?

Make sure you're using the proper gray codes for the frame buffer pointers. See appendix D:

- **Processing video frames using SDK and VDMA**

https://support.xilinx.com/s/question/0D52E00007G0fwqSAB/processing-video-frames-using-sdk-and-vdma?language=en_US

I've got a project which takes video frames from a camera, writing and reading them from/to a ddr3 memory using vdma triple buffer configuration with dynamic genlock setup for the write and read channels and a VGA output.

It is simple design but now I would like to do something with the video frames, for instance to apply some opencv functions, filters, color conversion...etc on SDK using C. Well, I guess that I can't operate on the vdma frames directly otherwise I would lose the synchronization....is it any example of video processing on sdk using similar configuration? what would be the best way to approach it?

[A] This use case is possible to do with the VDMA, but I don't know of any examples showing how to do it.

If it were me, I would use the external genlock feature of the VDMA to expose the genlock pointers. Then you could use a GPIO controlled by software to indicate to the VDMA which of its frames the processor is working on so that it will not overstep it.

- **VDMA Dynamic Genlock for pipelining frame capture (PL) and processing (PS)**

https://support.xilinx.com/s/question/0D74U000008loHISAK/detail?language=en_US&fromEmail=1&s1oid=00D2E000000nHq7&s1nid=0DB2E000000XdtH&s1uid=0052E00000N3Get&s1ext=0&emkind=chatterCommentNotification&emtm=1712072236440&emvtn=XrILtN2ZItlwolyYbVCZ3uiGuDqXldWOs8JBX.QjESo%3D

I need to process in the PS the frames coming from a MIPI camera. Frames are written to DDR by one VDMA channel at higher rate than the software processes them. In the current configuration, the camera is in free running mode and the VDMA is deactivated until the PS is ready to work. Then the PS activates the VDMA and wait until a new frame is written (interrupt flag). This way, the total time to do the task is the addition of capture (PL) and processing (PS) times.

I would like to pipelining both stages using a triple buffer and Dynamic GenLock mode, then a new frame is always available for PS processing without any delay.

To this end my proposal is to configure the VDMA write channel as Master Dynamic GenLock and the PS application as Slave, however it looks like the genlock mechanism is conceived to synchronize VDMA channels but not VDMA and PS apps.

How the VDMA previously written frame number can be known by the PS app?

How the PS app can communicate the frame number it is working on to the VDMA channel? are there special VDMA registers for that information?

Is there a best solution for VDMA and PS sync via triple-buffer?

Thanks

[A] The VFBW/VFBR gives you control of everything, where the VDMA is better suited for applications where you can set it up and let it run unattended, cycling through the frame buffers in a sequence controlled by the genlock mode you choose.

With the VFBW/VFBR IPs, you have complete control from your processor. You get both a read and write interrupt at the end of a frame, which allows you to control the frame synchronization and select which frame buffer to read or write.

If you want to set up your read side as the master, simply put the frame counter increment in the VFBR ISR, and read the current frame in the VFBW ISR. Set the new read and write frame buffer address in each respective ISR.

Below is an example of how to set up the VFBR interrupt handler as a master in a triple-buffer memory. The VFBW ISR can then read the value of the global rdFrmCtr variable to determine which frame is currently being read, so you can skip over that frame in your write ISR if you want the write side to be a dynamic slave.

```
static u8 rdFrmCtr = 0;

1.
2. void VFBR_InterruptHandler(void *CallbackRef)
3. {
4.     u32 bufferAddress;
5.
6.     rdFrmCtr = (rdFrmCtr == 2) ? 0 : rdFrmCtr + 1;
7.
8.     switch(rdFrmCtr) // set new read buffer address
9.     {
10.    case 0:
11.        bufferAddress = F1_BUFFER_ADDRESS; // frame 1 (id = 0)
12.        break;
13.    case 1:
14.        bufferAddress = F2_BUFFER_ADDRESS; // frame 2 (id = 1)
15.        break;
16.    case 2:
17.        bufferAddress = F3_BUFFER_ADDRESS; // frame 3 (id = 2)
18.        break;
19.    default:
20.        bufferAddress = F1_BUFFER_ADDRESS;
21.        break;
22.    }
23.
24.    XVFrmbufRd_SetBufferAddr(&VFBR, bufferAddress);
25.    XVFrmbufRd_Start(&VFBR);
26. }
```

5. Alternativas de implementación

1) Utilizando VDMA Triple buffer y Genlock dynamic

I would use the external genlock feature of the VDMA to expose the genlock pointers. Then you could use a GPIO controlled by software to indicate to the VDMA which of its frames the processor is working on so that it will not overstep it.

- Configurar el canal Write como Master Genlock Dynamic y en modo circular. Se crearán los puertos para los punteros de los ptrbuffer_out y ptrbuffer_in.

- Utilizar un GPIO controlado por el SW para escribir en el ptrbuffer_in e indicar al VDMA en cual de los frames está trabajando el SW. El VDMA seguirá trabajando con los otros dos frames.

- Cuando el SW quiera acceder a un nuevo frame, debe leer del registro PARK_PTR_REG el campo WriteFrameStore que indica en que buffer está trabajando actualmente el VDMA

2) Utilizando VFBW, Triple buffer y Genlock dynamic

I would recommend using the Video Frame Buffer Write and Video Frame Buffer Read IPs instead of the VDMA. You can keep track of the current frame number in the interrupt service routine for each IP, and control the master/slave behavior in SW.

En nuestro caso no es necesario un canal de lectura (VFBR):

- El Sw es el máster
- Sólo es necesario controlar las interrupciones del VFBW. El controlador actualizará cual es el último frame escrito e iniciará la escritura del siguiente frame teniendo en cuenta el buffer donde está trabajando el Sw.

- Cuando el Sw necesite procesar un nuevo frame, una función Sw se encargará de actualizar la dirección del Buffer de donde se debe leer (el buffer último escrito por el VFBW).

Propuesta de código:

```

u8 FirstFrmWrt=0;
u8 wrFrmCtr=0;
u8 Last_wrtFrmCtr;
u8 rdFrmCtr;
u8 rdActive=0;

main(){
    u32 rdBuffAdd;
    // Config
    XVFrmbufWr_SetBufferAddr(&VFBW, F1_BUFFER_ADDRESS);
    XVFrmbufWr_Start(&VFBW);
    while (!FirstFrmWrt); // wait until first frm written
    // ...
    do{
        rdBuffAdd = RdAddUpdate();
        Task1(rdBuffAdd);
        Task2(rdBuffAdd);
        Task3(rdBuffAdd);
        // ...
    }
    while(!EndCond);
}

u32 RdAddUpdate(){
    u32 bufferAddress;

    // disable VFBW interrupt
    rdFrmCtr = Last_wrtFrmCtr;
    // enable VFBW interrupt

    // set new read buffer address
    switch(rdFrmCtr){
        case 0:
            bufferAddress = F1_BUFFER_ADDRESS; // frame 1 (id = 0)
            break;
        case 1:
            bufferAddress = F2_BUFFER_ADDRESS; // frame 2 (id = 1)
            break;
        case 2:
            bufferAddress = F3_BUFFER_ADDRESS; // frame 3 (id = 2)
            break;
        default:
            bufferAddress = F1_BUFFER_ADDRESS;
            break;
    }
    return bufferAddress;
}

void VFBW_InterruptHandler(void *CallbackRef){
    u32 bufferAddress;

    FirstFrmWrt = 1;
    Last_wrtFrmCtr = wrFrmCtr;
    // set next wrFrmCtr
    if ( !rdActive){ // Circular buffer
        wrFrmCtr = (wrFrmCtr == 2) ? 0 : wrFrmCtr + 1;
    }
    else{
        // Skip rdFrm
        switch(rdFrmCtr){
            case 0:

```

```

        wrFrmCtr=(wrFrmCtr==1)? 2 : 1;
        break;
    case 1:
        wrFrmCtr=(wrFrmCtr==2)? 0 : 2;
        break;
    case 2:
        wrFrmCtr=(wrFrmCtr==0)? 1 : 0;
        break;
    default:
        wrFrmCtr=0;
}

// set new write buffer address
switch(wrFrmCtr)
{
    case 0:
        bufferAddress = F1_BUFFER_ADDRESS; // frame 1 (id = 0)
        break;
    case 1:
        bufferAddress = F2_BUFFER_ADDRESS; // frame 2 (id = 1)
        break;
    case 2:
        bufferAddress = F3_BUFFER_ADDRESS; // frame 3 (id = 2)
        break;
    default:
        bufferAddress = F1_BUFFER_ADDRESS;
        break;
}

XVFrmBufWr_SetBufferAddr(&VFBR, bufferAddress);
XVFrmBufWr_Start(&VFBR);
}

```

6. Análisis de proyectos de referencia

- Adam Taylor, código VFBRd VFBwr:

<https://www.adiuvoengineering.com/post/microzed-chronicles-video-frame-read-buffer>
<https://www.hackster.io/adam-taylor/ac701-image-processing-7ff161>

- Opsera, proyecto Vivado VFBRd VFBwr, código:

<https://rpi.camerafm.com/en/latest/>

- Xilinx Video Series34: getting started with the VFB IP, ejemplo básico basado en el ejemplo 2019.2

https://support.xilinx.com/s/article/1029060?language=en_US

Comenzar montando ejemplo de 2019.

Opción A:

a) Montar Kira-PCAM 2021.1 => poner en marcha => done

b) Completar con Vproc_sys => se cambió por IP Subset para hacer la conversión de forma más sencilla: RGB a BBB => done

c) Sustituir con VFBwr / VFBRd (código ejemplo xilinx, código ejemplo Adam Taylor):wq

6.1 Proyecto base Mipi VDMA (basado en Mipi Kria Adam Taylor)

HW

a) AXI Subset Converter

Component Name

axis_subset_converter_1

Slave Interface Signal Properties

AXIP0

Enable TREADY

Yes

▼

AXIP0

TDATA Width (bytes)

2

⊗

[0 - 512]

AXIP0

Enable TSTRB

No

▼

AXIP0

Enable TKEEP

No

▼

AXIP0

Enable TLAST

Yes

▼

AXIP0

TID Width (bits)

0

⊗

[0 - 32]

AXIP0

DEST Width (bits)

10

⊗

[0 - 32]

AXIP0

USER Width (bits)

1

⊗

[0 - 4096]

Master Interface Signal Properties

AXIP0

Enable TREADY

Yes

▼

AXIP0

TDATA Width (bytes)

1

⊗

[0 - 32]

AXIP0

Enable TSTRB

No

▼

AXIP0

Enable TKEEP

No

▼

AXIP0

Enable TLAST

Yes

▼

AXIP0

TID Width (bits)

0

⊗

[0 - 32]

AXIP0

DEST Width (bits)

10

⊗

[0 - 32]

AXIP0

USER Width (bits)

1

⊗

[0 - 4096]

Extra Settings

TDATA Remap String

[data[9:2]

Component Name

v_demosaic_c_0

Samples per Clock

1

▼

Maximum Data Width

8

▼

Maximum Number of Columns

2048

⊗

[64 - 8192]

Maximum Number of Rows

1024

⊗

[64 - 4320]

Interpolation Method

High Resolution Interpolation

▼

☑ Horizontal Zipper Artifact Removal

Use UltraRAM for Line Buffers

Component Name		v_gamma_lut_0	
Samples per Clock	1		
Maximum Data Width	8		
Maximum Number of Columns	2048		[64 - 8192]
Maximum Number of Rows	1024		[64 - 4320]

Component Name

axis_subset_converter_2

Slave Interface Signal Properties

☐ AIFD

Enable TREADY

Yes

▼

☐ TDATA

TDATA Width (bytes)

3

Ⓢ

[0 - 512]

☐ AIFD

Enable TSTRB

Yes

▼

☐ TDATA

Enable TKEEP

Yes

▼

☐ TDATA

Enable TLAST

Yes

▼

☐ AIFD

TID Width (bits)

1

[0 - 32]

☐ AIFD

DEST Width (bits)

1

[0 - 32]

☐ TDATA

USER Width (bits)

1

Ⓢ

[0 - 4096]

Master Interface Signal Properties

☐ AIFD

Enable TREADY

Yes

▼

☐ TDATA

TDATA Width (bytes)

3

Ⓢ

[0 - 512]

☐ AIFD

Enable TSTRB

Yes

▼

☐ TDATA

Enable TKEEP

Yes

▼

☐ TDATA

Enable TLAST

Yes

▼

☐ AIFD

TID Width (bits)

1

[0 - 32]

☐ AIFD

DEST Width (bits)

1

[0 - 32]

☐ TDATA

USER Width (bits)

1

Ⓢ

[0 - 4096]

Extra Settings

TDATA Domain String

[data1[7:0] data1[7:0] data1[7:0]

Component Name

axi_vdma_0

Basic

Advanced

Address Width (32-64)

32

bits

Frame Buffers

4

☒ Enable Write Channel

auto

Memory Map Data Width

64

Write Burst Size

8

Stream Data Width (Auto)

24

Line Buffer Depth

512

☒ Enable Read Channel

64

Memory Map Data Width

8

Read Burst Size

24

Stream Data Width

512

Line Buffer Depth

Component Name

axi_vdma_0

Basic

Advanced

☐ Enable Asynchronous Mode (Auto)

auto

When checked, AXI VDMA will be configured to operate in asynchronous clocks.

☐ Enable Vertical Flip

Write Channel Options

Fsync Options

s2mm tuser

GenLock Mode

Dynamic-Master

☐ Allow Unaligned Transfers

Read Channel Options

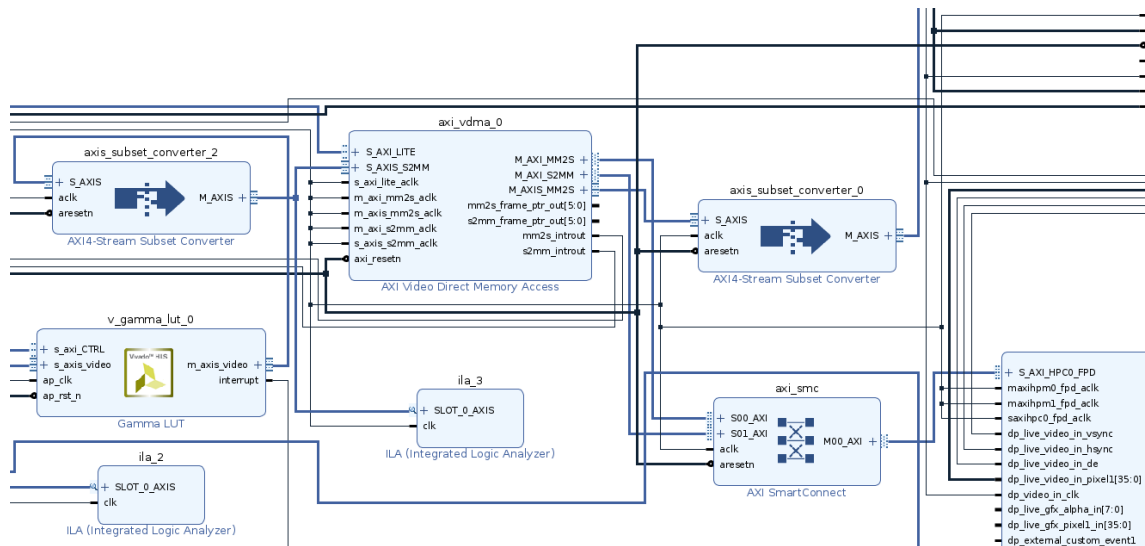
Fsync Options

None

GenLock Mode

Dynamic-Slave

☐ Allow Unaligned Transfers



Interrupciones VDMA conectadas pero no utilizadas

SW

Utiliza el mismo buffer para escribir y leer

```
XAxiVdma vdma;
XAxiVdma_DmaSetup vdmaDMA;
XAxiVdma_Config *vdmaConfig;

#define DEMO_MAX_FRAME (720*1280)
#define DEMO_STRIDE (1280*3)
#define DISPLAY_NUM_FRAMES 1
u32 frameBuf[DISPLAY_NUM_FRAMES][DEMO_MAX_FRAME];
u32 *pFrames[DISPLAY_NUM_FRAMES];
```

```

int main(){
    //init_platform
    //detect_camera
    //set up the video timing controller

    pFrames[i] = frameBuf[i];
    //Initialize VDMA
    vdmaConfig = XAxiVdma_LookupConfig(XPAR_AXIVDMA_0_DEVICE_ID);
    XAxiVdma_CfgInitialize(&vdma, vdmaConfig, vdmaConfig->BaseAddress);

    //video = VMODE_1280x720;
    vdmaDMA.FrameDelay = 0;
    vdmaDMA.EnableCircularBuf = 1;
    vdmaDMA.EnableSync = 0;
    vdmaDMA.PointNum = 0;
    vdmaDMA.EnableFrameCounter = 0;
    vdmaDMA.VertSizeInput = video.height;
    vdmaDMA.HoriSizeInput = (video.width)*3;
    vdmaDMA.FixedFrameStoreAddr = 0;
    vdmaDMA.FrameStoreStartAddr[0] = (u32) pFrames[0];
    vdmaDMA.Stride = (video.width)*3;

    //Configure VDMA Write Channel
    XAxiVdma_DmaConfig(&vdma, XAXIVDMA_WRITE, &(vdmaDMA));
    Status = XAxiVdma_DmaSetBufferAddr(&vdma,
XAXIVDMA_WRITE, vdmaDMA.FrameStoreStartAddr);
    //Configure VDMA Read Channel
    XAxiVdma_DmaConfig(&vdma, XAXIVDMA_READ, &(vdmaDMA));
    XAxiVdma_DmaSetBufferAddr(&vdma,
XAXIVDMA_READ, vdmaDMA.FrameStoreStartAddr);

    // Configure Demosaic, gamma, etc..

    //Start Write Channel
    Status = XAxiVdma_DmaStart(&vdma, XAXIVDMA_WRITE);
    Status = XAxiVdma_StartParking(&vdma, 0, XAXIVDMA_WRITE);
    //Start Read Channel
    XAxiVdma_DmaStart(&vdma, XAXIVDMA_READ);
    XAxiVdma_StartParking(&vdma, 0, XAXIVDMA_READ);

    run_dpdsu(); //Inicial el DP

    while(1){

```

6.2 Proyecto FrmBuf_Wr Adam Taylor

TPG => FrmBuf_Wr => DDR

- a) **Reset** FrmBuf_Wr conectado al *Procesor System Reset::peripheral_aresetn*
- b) auto-restart mode

1. Initialize, configure, and start the test pattern generator
2. Initialize the Video Frame Buffer Write

```
XV_frmbufwr_Initialize(&frmbufwr, XPAR_V_FRMBUF_WR_0_DEVICE_ID);
```

3. Configure the Video Frame Buffer Write

```

XV_frmbufwr_Set_HwReg_width(&frmbufwr, 1920);
XV_frmbufwr_Set_HwReg_height(&frmbufwr, 1080);
XV_frmbufwr_Set_HwReg_stride(&frmbufwr, 4*1920);

```

```

        XV_frmbufwr_Set_HwReg_video_format(&frmbufwr,
XVIDC_CSF_MEM_RGB8);
        XV_frmbufwr_Set_HwReg_frm_buffer_V(&frmbufwr,
XVFRMBUFWR_BUFFER_BASEADDR);

```

4. Start the Video Frame Buffer Write

```

        XV_frmbufwr_EnableAutoRestart(&frmbufwr);
        XVFrmbufWr_Start(&frmbufwr);

```

6.3 Proyecto FrmBuf_Rd Adam Taylor

MIPI => FrmBuf_Wr => DDR
 DDR => FrmBuf_Rd => Vproc_ss => AXISstream to Video Out

- Initialize the VFBR
- Set the image width and height, Set the image stride (this is the image width multiplied by the bytes per pixel), Set the video format, Set the frame buffer address
- Start the VFBR

Utiliza el mismo BUFF para escribir y leer. No utiliza int

```

#define PAGE_SIZE    16

#include "xv_frmbufwr.h"
#include "xv_frmbufrd.h"
#define DDR_BASEADDR XPAR_MIG7SERIES_0_BASEADDR
#define XVFRMBUFWR_BUFFER_BASEADDR (DDR_BASEADDR + (0x1000000))

XV_frmbufwr      frmbufwr;
XV_frmbufwr_Config frmbufwr_cfg;

XV_frmbufrd frmbufrd;
XV_frmbufrd_Config frmbufrd_cfg;

int main()
{

    // Configure camera, I2c and VTC
    // Enables the sensors.

    XV_frmbufwr_Initialize(&frmbufwr,  XPAR_V_FRMBUF_WR_0_DEVICE_ID);
    XV_frmbufwr_Set_HwReg_width(&frmbufwr, 1920);
    XV_frmbufwr_Set_HwReg_height(&frmbufwr, 1080);
    XV_frmbufwr_Set_HwReg_stride(&frmbufwr, 4*1920);
    XV_frmbufwr_Set_HwReg_video_format(&frmbufwr, XVIDC_CSF_MEM_RGB8);
    XV_frmbufwr_Set_HwReg_frm_buffer_V(&frmbufwr,
XVFRMBUFWR_BUFFER_BASEADDR);
    XV_frmbufwr_EnableAutoRestart(&frmbufwr);
    XV_frmbufwr_Start(&frmbufwr);

    XV_frmbufrd_Initialize(&frmbufrd, XPAR_V_FRMBUF_RD_0_DEVICE_ID);
    XV_frmbufrd_Set_HwReg_width(&frmbufrd, 1920);
    XV_frmbufrd_Set_HwReg_height(&frmbufrd, 1080);
    XV_frmbufrd_Set_HwReg_stride(&frmbufrd, 4*1920);
    XV_frmbufrd_Set_HwReg_video_format(&frmbufrd, XVIDC_CSF_MEM_RGB8);
    XV_frmbufrd_Set_HwReg_frm_buffer_V(&frmbufrd,
XVFRMBUFWR_BUFFER_BASEADDR);
    XV_frmbufrd_EnableAutoRestart(&frmbufrd);

```

```

    XV_frmbufprd_Start(&frmbufprd);

    while(1){ }
}

```

6.4 Proyecto Opsero

HW

```

/*
 * Opsero Electronic Design Inc. Copyright 2023
 *
 * This example standalone application for the RPi Camera FMC will configure all of the connected
 * cameras that it finds connected, and then displays the video outputs on the DisplayPort monitor.
 * To display all four video streams on the single display, it uses the Video Processor Subsystem IP
 * to downsize the 1080p videos coming from the cameras to 720x480 resolution and then combines them
 * with the Video Mixer to produce a 1080p resolution video containing all four video streams.
 * The camera streams are organized on the screen as shown below:
 *
 *
 * +-----+
 * |               1920x1080px               |
 * |  +-----+ +-----+  |
 * |  | CAM0   | | CAM1   |  |
 * |  | 720x480px | | 720x480px |  |
 * |  +-----+ +-----+  |
 * |  +-----+ +-----+  |
 * |  | CAM2   | | CAM3   |  |
 * |  | 720x480px | | 720x480px |  |
 * |  +-----+ +-----+  |
 * +-----+
 *
 * Camera-to-DDR video pipe (x4):
 * -----
 * RPi cam -> MIPI CSI SS -> Demosaic -> Gamma LUT -> VProc SS -> Frame Buffer Write -> DDR
 *
 * The video pipe runs at 2 pixels per clock.
 *
 * DDR-to-DP video pipe:
 * -----
 *
 * Video Test Pattern Generator -> |
 * DDR -> Frame Buffer Read (CAM0) -> |
 * DDR -> Frame Buffer Read (CAM1) -> | Video Mixer | -> AXIS Remapper -> AXIS to Video Out -> DP
live
 * DDR -> Frame Buffer Read (CAM2) -> | (2ppc) | (2ppc to 1ppc)
 * DDR -> Frame Buffer Read (CAM3) -> |
 *
 * Input and output of the video mixer runs at 2 pixels per clock. The AXIS Remapper converts
 * the 2ppc to 1ppc before it gets converted to video signals for the ZynqMP's DP live interface.
 */

```

- Las interrupciones de VFB_rd y VFB_wr se conectan directamente al puerto pl_ps_irq0
- Los IPs trabajan en modo auto-restart
- Se configuran los ISRs de los IPs.

SW

El proyecto define un triple (cuádruple?) buffer circular para sincronizar la escritura y la lectura de imágenes.

```

/* generado con https://asciiflow.com/#/
*
* FRMBUF_BUFR_ADDR+-----+ ^
* (0x1000_0000) |   BUF0   | | FRAME_SIZE
*               |-----+ | (0x0100_0000)
*               |   BUF1   | |
*               |-----+ |
*               |   BUF2   | |
*               |-----+ |
*               |   BUF3   | |
*               |-----+ |
*
*/

```

```

// main.c
// - define la dirección del Video Buffer CAM0_FRMBUF_BUFR_ADDR
// - configura las interrupciones (Gic)
// - inicializa el videopipe mediante las struct
// - arranca la aplicación DP

#include "xscugic.h"
...
#include "xavbuf.h"
#include "xv_frmbufrd_l2.h"
#include "xv_frmbufwr_l2.h"
...
#include "xdpdma_video_example.h"
#include "board.h"
#include "pipe.h"
#include "config.h"

// Frame buffer Rd/Wr buffer addresses (using physical memory)
#define CAM0_FRMBUF_BUFR_ADDR 0x10000000
#define CAM1_FRMBUF_BUFR_ADDR 0x20000000
#define CAM2_FRMBUF_BUFR_ADDR 0x30000000
#define CAM3_FRMBUF_BUFR_ADDR 0x40000000

XScuGic Intc;

VideoPipe Cam0;

// Provide the device IDs for the elements of each video pipe
VideoPipeDevIds CamDevIds0 = {
    XPAR_MIP1_0_AXI_IIC_0_DEVICE_ID,
    XPAR_MIP1_0_AXI_GPIO_0_DEVICE_ID,
    XPAR_MIP1_0_V_FRMBUF_WR_DEVICE_ID,
    XPAR_MIP1_0_V_FRMBUF_RD_DEVICE_ID,
    CAM0_FRMBUF_BUFR_ADDR,
    XPAR_MIP1_0_DEMOSAIC_0_DEVICE_ID,
    XPAR_MIP1_0_V_GAMMA_LUT_DEVICE_ID,
    XPAR_MIP1_0_V_PROC_DEVICE_ID,
    XPAR_FABRIC_MIP1_0_AXI_IIC_0_IIC2INTC_IRPT_INTR,
    XPAR_FABRIC_MIP1_0_V_FRMBUF_WR_INTERRUPT_INTR,
    XPAR_FABRIC_MIP1_0_V_FRMBUF_RD_INTERRUPT_INTR
};

// Common target device IDs
#define INTC_DEVICE_ID XPAR_SCUGIC_SINGLE_DEVICE_ID

// Interrupt controller defines
// #define INTC_DEVICE_ID XPAR_SCUGIC_SINGLE_DEVICE_ID
#define INTC_HANDLER XScuGic_InterruptHandler

int main()
{
    XScuGic_Config *IntcConfig;

    /* Initialize the interrupt controller
     * The init functions of the I2C driver and DPDMA driver will each setup and
     * enable their respective interrupts later. */

    IntcConfig = XScuGic_LookupConfig(INTC_DEVICE_ID);

    Status = XScuGic_CfgInitialize(&Intc, IntcConfig, IntcConfig->CpuBaseAddress);

```

```

// Initialize exceptions
Xil_ExceptionInit();
Xil_ExceptionRegisterHandler(XIL_EXCEPTION_ID_IRQ_INT,
                             (Xil_ExceptionHandler)XScuGic_DeviceInterruptHandler,
                             INTC_DEVICE_ID);
// Enable exceptions for interrupts
Xil_ExceptionEnableMask(XIL_EXCEPTION_IRQ);
Xil_ExceptionEnable();
/* Initialize Video pipe for each camera */
NumActiveCams = 0;
#ifdef XPAR_MIPID0_AXI_IIC0_DEVICE_ID
    pipe_init(&Cam0, &CamDevIds0, &Intc);

    /* Run the DisplayPort video example*/
    Xil_DCacheDisable();
    Xil_ICacheDisable();

    Status = DpdmaVideoExample(&RunCfg,&Intc,Stream.VmId);
    /* Start the connected cameras and enable corresponding mixer layers */
    for(int i = 0; i < NumActiveCams; i++) {
        pipe_start_camera(ActiveCams[i]);
        XVMix_LayerEnable(&VMix, i+1);
    }

    while(1){
    }
}

// pipe.c
// - configura el videopipe de adquisición y preproc de imágenes mediante la struct VideoPipe
// CamX-> MIPI -> Demosaic -> Gamma -> VProc -> Frmbuf Write -> DDR
// DDR -> Frmbuf Read (CAMX) -> VideoMixer->AXIStoVideoOut-> DP

#include "xv_frmbufwr_l2.h"
#include "xv_frmbufwr_l2.h"
// en el .h #include "xscugic.h" #include "frmbuf.h"
// VideoPipe struct
typedef struct { // SC: se pasa a pipe_init y este a su vez lo pasa a los inicializadores
    uint8_t IsConnected;
    XIic Iic; // handler se inicializa con IicAxiInit
    XGpio Gpio; // handler se inicializa con XGpio_Initialize
    XV_demosaic Demosaic; // handler se inicializa con
    XV_gamma Lut GammaLut; // handler se inicializa con
    XVprocSs Vproc; // handler se inicializa con
    RpiCamera Camera;
    Frmbuf Frmbuf; // incluye los handler de Fbwr y FBRd se inicializa con FrmbufWrInit y
    FrmbufRdInit
} VideoPipe; // Cam0, Cam1, ...

// VideoPipe device IDs
typedef struct { // SC: se rellena en el main
    u16 Iic;
    u16 Gpio;
    u16 FrmbufWr;
    u16 FrmbufRd;
    u32 FrmbufBufRBaseAddr; // SC: CAMX_FRMBUF_BUFR_ADDR
    u16 Demosaic;
    u16 GammaLut;
    u16 Vproc;
    u16 IicIntr;
    u16 FrmbufWrIntr;
    u16 FrmbufRdIntr;
} VideoPipeDevIds;

int pipe_init(VideoPipe *pipe, VideoPipeDevIds *devids, XScuGic *intc){

    /* Frame Buffer Wr/Rd initialization and config */
    // SC: se inicializan con el BaseAddr
    Status = FrmbufWrInit(&(pipe->Frmbuf),devids->FrmbufWr,intc,devids->FrmbufWrIntr,devids->FrmbufBufRBaseAddr);

    Status = FrmbufRdInit(&(pipe->Frmbuf),devids->FrmbufRd,intc,devids->FrmbufRdIntr,devids->FrmbufBufRBaseAddr);
}

int pipe_start_camera(VideoPipe *pipe){
    int Status;

    // Start the RPi camera
    Status = rpi_cam_config(&(pipe->Camera));
}

```

```

    // Start the Frame buffers
    Status = FrmbufStart(&(pipe->Frmbuf));
    return(Status);
}

// frmbuf.c
// - define las funciones para manejar los IPs: FrmbufWrInit, FrmbufRdInit
// - utiliza la struct Frmbuf (incluida en struct VideoPipe)
// - define las Callback (ISR): actualizan las direcciones de Rd y Wr de las imágenes.
// Se utiliza un triple (cuádruple?) buffer circular de forma que el el canal Rd lee del último
// buffer escrito por Wr, mientras Wr escribe el siguiente. La actualización de las direcciones de
// lectura y escritura se realiza cada vez que salta la int de Wr (en la ISR del Wr).
//

// en el .h #include "xparameters.h" #include "xv_frmbufwr_l2.h" #include "xv_frmbufwr_l2.h"
// #include "xil_exception.h" #include "xscugic.h"
// typedef struct { // SC: se rellena con FrmbufWrInit y FrmbufRdInit
//     XV_FrmbufWr_l2 FrmbufWr; // handler se inicializa con XVFrmbufWr_Initialize
//     XV_FrmbufRd_l2 FrmbufRd; // handler se inicializa con XVFrmbufRd_Initialize
//     u64 FrmbufBufRBaseAddr; // se rellena con FrmbufBufRBaseAddr = CAMX_FRMBUF_BUFR_ADDR
//     u64 FrmbufWrBufRAddr; // se rellena con FrmbufBufRBaseAddr = CAMX_FRMBUF_BUFR_ADDR
//     u64 FrmbufRdBufRAddr; // se rellena con FrmbufBufRBaseAddr = CAMX_FRMBUF_BUFR_ADDR
//     // WrBufRAddr y RdBufRAddr se van cambiando en la ISR FrmbufWrCallback
// } Frmbuf;

/* Initialize Frame Buffer Write
 * - Enable interrupts
 * - Assign the callback function
 * - Enable auto-restart mode
 * - Sets the stream parameters to align with the VProc SS output stream */

int FrmbufWrInit(Frmbuf *Frmbuf, u16 DeviceId, XScuGic *Intc, u16 VectorId, u64 BufRBaseAddr){
    // Hook up interrupt service routine
    Status = XScuGic_Connect(Intc, VectorId,
        (XInterruptHandler)XVFrmbufWr_InterruptHandler,
        (void *)&(Frmbuf->FrmbufWr));

    // Enable the interrupt vector at the interrupt controller
    XScuGic_Enable(Intc, VectorId);

    // Init Frame buffers
    Status = XVFrmbufWr_Initialize(&(Frmbuf->FrmbufWr), DeviceId);

    // Assign the Frame buffer callbacks
    XVFrmbufWr_SetCallback(&(Frmbuf->FrmbufWr), XVFRMBUFWR_HANDLER_DONE, FrmbufWrCallback,
        (void *)Frmbuf);

    // Configure Frame Buffers
    // SC:
    Frmbuf->FrmbufWrBufRAddr = BufRBaseAddr;
    Frmbuf->FrmbufBufRBaseAddr = BufRBaseAddr;
    Status = XVFrmbufWr_SetMemFormat(&(Frmbuf->FrmbufWr), FrmbufWrStride, Cfmt,
    &FrmbufWrStream);

    Status = XVFrmbufWr_SetBufferAddr(&(Frmbuf->FrmbufWr), Frmbuf->FrmbufWrBufRAddr);

    // Enable Interrupt
    XVFrmbufWr_InterruptEnable(&(Frmbuf->FrmbufWr), XVFRMBUFWR_IRQ_DONE_MASK);

    // Frame Buffers in Auto-restart mode
    XV_frmbufwr_EnableAutoRestart(&(Frmbuf->FrmbufWr.FrmbufWr));
}

int FrmbufStart(Frmbuf *Frmbuf){
    XVFrmbufWr_Start(&(Frmbuf->FrmbufWr));
    XVFrmbufRd_Start(&(Frmbuf->FrmbufRd));
}

#define FRAME_SIZE 0x01000000
// Callback (ISR) functions
void *FrmbufWrCallback(void *data)
{
    Frmbuf *frmbuf = (Frmbuf *)data;
    XV_FrmbufWr_l2 *frmbufwr = &(frmbuf->FrmbufWr);
    XV_FrmbufRd_l2 *frmbufrd = &(frmbuf->FrmbufRd);

    // Rotate the Read and Write Frame Buffer addresses
    if(frmbuf->FrmbufWrBufRAddr >= (frmbuf->FrmbufBufRBaseAddr + (FRAME_SIZE * 3))){

```



```

        frmbuf->FrmbufRdBufrAddr = frmbuf->FrmbufBufrBaseAddr + (FRAME_SIZE * 2);
        frmbuf->FrmbufWrBufrAddr = frmbuf->FrmbufBufrBaseAddr + (FRAME_SIZE);
    }else{
        frmbuf->FrmbufRdBufrAddr = frmbuf->FrmbufWrBufrAddr;
        frmbuf->FrmbufWrBufrAddr = frmbuf->FrmbufWrBufrAddr + FRAME_SIZE;
    }

    // Set the buffer address to write the next frame
    Status = XVFrmbufWr_SetBufferAddr(frmbufwr, frmbuf->FrmbufWrBufrAddr);

    // Set the buffer address to read the next frame
    Status = XVFrmbufRd_SetBufferAddr(frmbufrd, frmbuf->FrmbufRdBufrAddr);
}

void *FrmbufRdCallback(void *data)
{
    Frmbuf *frmbuf = (Frmbuf *)data;
    XV_FrmbufRd_l2 *frmbufrd = &(frmbuf->FrmbufRd);
    // Nothing to do - Frame Buffer Read is configured for auto-restart
}

```

Funcionamiento:

- Inicialmente `FrmbufWrBufrAddr` y `FrmbufRdBufrAddr` apuntan a `CAMX_FRMBUF_BUFR_ADDR`.
- Cuando VBWr escribe la primera imagen salta la `intWr` y cambia los punteros a los buffer de Wr y de Rd y se VBWr se resetea para comenzar una nueva captura en un buffer distinto.
- Cuando VBRd termina la lectura salta la `IntRd` y simplemente se resetea para comenzar una nueva lectura desde el buf nuevo.

WARNING: con esta lógica, cuando la Wr es más rápida que la Rd, se puede producir colisión en el buffer

TODO: Opsero modifica el `dpdma_video_example.c` para que no trabaje con graphics overlay

7. Generación proyecto base mipi + VFB

7.1 Adaptación proyecto Mipi VDMA (Adam Taylor Mipi Kria)

Paso_1: Añadir VFB_Wr

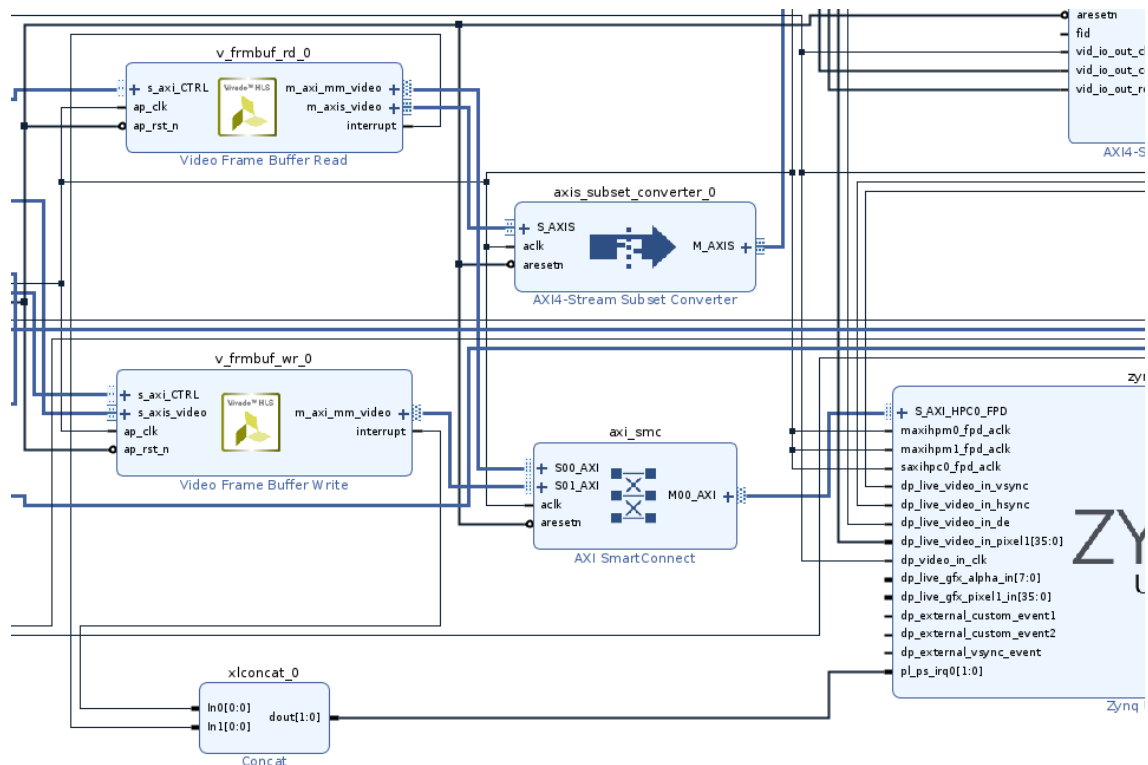
ap_rst_n=> peripheral_areset_n; ap_clk=>video_aclk(150MHz);

Paso_2: Añadir VFB_Rd:

ap_rst_n=> peripheral_areset_n; ap_clk=>video_aclk(150MHz);

Paso_3: Eliminar AXI Interrupt Controller. Concatenar wr_irq/rd_irq y conectar al puerto UltraScale pl_ps_irq0[1:0] (conexión al Global Interrupt Controller GIC)

nombre de la plataforma **mipi_frmbuf_plat**



Component Name: **v_frmbuf_rd_0**

Samples per Clock: 1

Maximum Number of Columns: 2048 [64 - 10328]

Maximum Number of Rows: 1024 [64 - 7760]

Maximum Data Width: 8

Address Width: 32

☐ Video Formats with Alpha

☐ Interlaced Support

Video Formats with per pixel Alpha

☐ RGBA8 ☐ BGRA8

☐ YUV48

8 Bit Video Formats

☐ RGBX8 ☒ RGB8 ☐ BGRX8 ☐ BGR8

Component Name: **v_frmbuf_wr_0**

Samples per Clock: 1

Maximum Number of Columns: 2048 [64 - 10328]

Maximum Number of Rows: 1024 [64 - 7760]

Maximum Data Width: 8

Address Width: 32

☐ Interlaced Support

8 Bit Video Formats

☐ RGBX8 ☒ RGB8 ☐ BGRX8 ☐ BGR8

7.2 Adaptación del Sw

Se han creado 3 aplicaciones:

- **mipi_frmbuf_l2_app**: utiliza los drivers de nivel L2 para configurar los IPs, trabaja sin interrupciones como el proyecto original y un único buffer donde se solapan Wr y Rd.

- **mipi_frmbuf_int_test_app**: interrupciones de los IPs habilitadas, las funciones de Callback sólo envían un mensaje confirmando la generación de la int. *Importante*: es necesario modificar las funciones xpdma_video_example para que utilicen el mismo manejador del GIC que se utilizó en el main

- **mipi_frmbuf_int_trbuf_app**: interrupciones de los IPs habilitadas, las funciones de Callback controlan un Triple Buffer para no solapar Wr y Rd.

Estructura básica del SW para trabajar con los IPs FrmBuffer y sus interrupciones

```
// Funciones drivers nivel L1: XV_XXXXX
// Funciones drivers nivel L2: XVxxx

/* 1. INICIALIZACIÓN Y CONFIGURACIÓN DEL CONTRL. DE INTR. GIC */

// a) Genera el manejador para configurar el GIC
IntcConfig = XScuGic_LookupConfig(XPAR_SCUGIC_0_DEVICE_ID);

// b) Inicializa excepciones
Xil_ExceptionInit();

Xil_ExceptionRegisterHandler(XIL_EXCEPTION_ID_IRQ_INT,
(Xil_ExceptionHandler)XScuGic_DeviceInterruptHandler, INTC_DEVICE_ID);

// c) Habilita las excepciones para las intr
Xil_ExceptionEnableMask(XIL_EXCEPTION_IRQ);
Xil_ExceptionEnable();

/* 2. CONFIGURACIÓN DE INTR DE LOS IPs */

// a) Incluye el IntHandler (ISR) del IP en la Tabla de vectores de int (Handler Table).
// Este IntHandler está definido en los drivers de nivel L2 y hace lo siguiente:
// - comprueba el flag de interrupción (ap_done)
// - si hay intr :
// + limpia el bit
// + llama a la función de Callback

Status = XScuGic_Connect(&Intc, XPAR_FABRIC_V_FRMBUF_WR_0_INTERRUPT_INTR,
(XInterruptHandler) XVFrmbufWr_InterruptHandler,
(void *) &frmbufwr);
// b) Habilita la conexión del GIC con la int del IP
XScuGic_Enable(&Intc, XPAR_FABRIC_V_FRMBUF_WR_0_INTERRUPT_INTR);
// c) Define la función de Callback asociada a la int. del IP
XVFrmbufWr_SetCallback(Frmbuf.FrmbufWrPtr, XVFRMBUFWR_HANDLER_DONE, FrmbufWrCallback, (void *) &Frmbuf);
// d) Habilita la int. del IP
XVFrmbufWr_InterruptEnable(&frmbufwr, XVFRMBUFWR_IRQ_DONE_MASK);

/* 3. CONFIGURACIÓN DE LOS IPs */

// a) Inicializa el manejador del IP
Status = XVFrmbufWr_Initialize(&frmbufwr, XPAR_V_FRMBUF_WR_0_DEVICE_ID);
// b) Configura los registros del IP (características del stream de video)
cfrmbufwr_Set_HwReg_width(&(frmbufwr.FrmbufWr), 1280);
XV_frmbufwr_Set_HwReg_height(&(frmbufwr.FrmbufWr), 720);
XV_frmbufwr_Set_HwReg_stride(&(frmbufwr.FrmbufWr), 3*1280);
XV_frmbufwr_Set_HwReg_video_format(&(frmbufwr.FrmbufWr), XVIDC_CSF_MEM_RGB8);
// función equivalente L2: XVFrmbufWr_SetMemFormat()

// c) Configura la dirección del buffer de escritura
XV_frmbufwr_Set_HwReg_frm_buffer_V(&(frmbufwr.FrmbufWr), XVFRMBUFWR_BUFFER_BASEADDR);
// función equivalente L2: XVFrmbufWr_SetBufferAddr()

// d) Habilita el modo auto-restart. El IP se reinicia después de cada transacción
XV_frmbufwr_EnableAutoRestart(&(frmbufwr.FrmbufWr));

/* 4. ARRANQUE DE LOS IPs */

XV_frmbufwr_Start(&(frmbufwr.FrmbufWr));
```

```

/* 5. EJEMPLO FUNCIÓN DE CALLBACK */
void *FrmBufWrCallback(void *data)
{
    xil_printf("Finalizada transacción de escritura \r\n");
}

```

Depuración y test del proyecto mipi_frmbuf_int_test_app

En este caso seguimos teniendo un único buffer en memoria donde ambos IPs (FrmBuf_Wr y FrmBuf_Rd) acceden. Además cada IP genera una int. cuando finaliza su transacción y se auto-arranca para la siguiente.

- Para manejar las int. se utiliza el controlador de interrupciones de la PS: Global Interrupt Controller (GIC).
- Para notificar la int. se genera un mensaje a través de la UART0 en la función de CallBack de cada ISR.

Para generar este código se han combinado los proyectos:

- Mipi Kria Adam Taylor: pipeline de captura + VDMA + pipe de visualización en DisplayPort
- Opsero (multi-cam bare-metal): pipeline de captura FrmBufWr_L2+ FrmBufRd_L2 pipe de visualización en DisplayPort + configuración del GIC para los IPs FrmBuf, ejemplo de funciones de CallBack.
- Xilinx example frmbufrd: pipeline de escritura en memoria FrmBufWr_L2+ FrmBufRd_L2 pipe de lectura sin visualización + configuración del GIC para los IPs FrmBuf, ejemplo de funciones de CallBack.

Problema:

La primera versión del código (mipi_frmbuf_in_app) funciona correctamente la captura y visualización pero no saltan las interrupciones. Los IPs funcionan con auto-arranque cuando finaliza su transacción y generan el bit de interrupción (ap_done), sin embargo, la interrupción no es capturada por el GIC y no se ejecuta la ISR ni la función del CallBack. Claramente es un fallo de configuración del GIC.

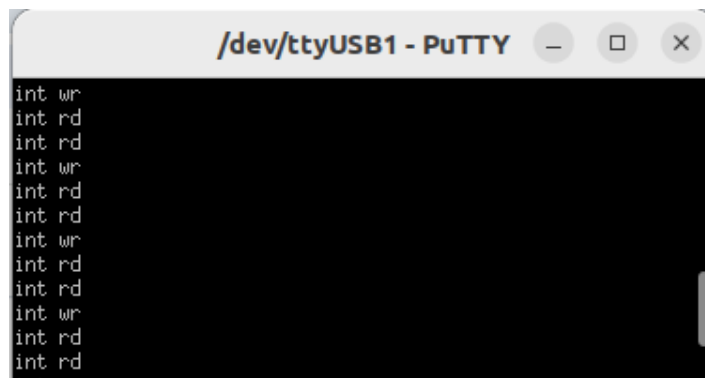
Solución:

La configuración del GIC se ha revisado y es correcta. El problema reside en la parte de la aplicación que configura el DP. Este código proviene del ejemplo de Xilinx que a su vez configura el GIC para las interrupciones del DP. Esta segunda configuración del GIC utiliza un nuevo manejador anulando el primero, por tanto las intr. de los IPs FrmBuf quedan desconectadas. Es necesario retocar el código del DP para que utilice el mismo manejador del GIC que se utilizó para los IPs.

Se añadió el puntero al manejador (XScuGic) como un parámetro que pasamos a la aplicación DP:

```
int DpdmaVideoExample(Run_Config *RunCfgPtr,XScuGic *Intr,XVidC_VideoMode mode);
```

Como puede apreciarse se notifican 2 lecturas por cada 1 escritura, o cual es compatible con los framerate de la cámara (30fps) y del modo de visualización del DP (60fps)



Depuración y test del proyecto mipi_frmbuf_int_trbuf_app:

Básicamente se modifican las funciones de CallBack de las interrupciones de los IPs para controlen un triple buffer circular, de forma que cada canal trabaje sobre un buffer distinto.

Código Opsero:

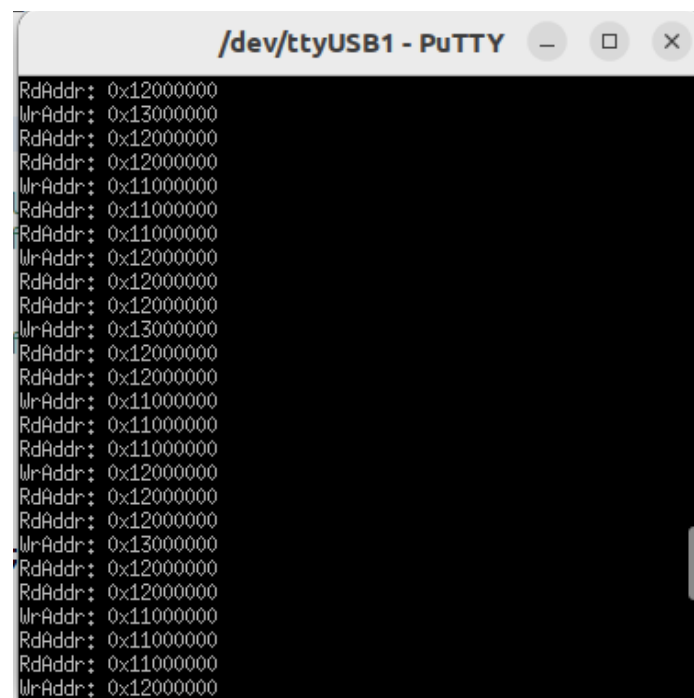
```
void *FrmbufWrCallback(void *data)
{
    Frmbuf_t *frmbuf = (Frmbuf_t *)data;
    XV_FrmbufWr_l2 *frmbufwr = frmbuf->FrmbufWrPtr;
    XV_FrmbufRd_l2 *frmbufrd = frmbuf->FrmbufRdPtr;

    xil_printf("WrAddr: 0x%x\r\n", frmbuf->FrmbufWrBufAddr);

    // Rotate the Read and Write Frame Buffer addresses
    if(frmbuf->FrmbufWrBufAddr >= (frmbuf->FrmbufBufBaseAddr + (FRAME_SIZE * 3))){
        frmbuf->FrmbufRdBufAddr = frmbuf->FrmbufBufBaseAddr + (FRAME_SIZE * 2);
        frmbuf->FrmbufWrBufAddr = frmbuf->FrmbufBufBaseAddr + (FRAME_SIZE);
    }else{
        frmbuf->FrmbufRdBufAddr = frmbuf->FrmbufWrBufAddr;
        frmbuf->FrmbufWrBufAddr = frmbuf->FrmbufWrBufAddr + FRAME_SIZE;
    }

    // Set the buffer address to write the next frame
    Status = XVFrmbufWr_SetBufferAddr(frmbufwr, frmbuf->FrmbufWrBufAddr);
    // Set the buffer address to read the next frame
    Status = XVFrmbufRd_SetBufferAddr(frmbufrd, frmbuf->FrmbufRdBufAddr);
}
```

El resultado es:



The screenshot shows a PuTTY terminal window titled "/dev/ttyUSB1 - PuTTY". The terminal displays a series of hexadecimal addresses for read and write operations. The sequence of addresses shown is: RdAddr: 0x12000000, WrAddr: 0x13000000, RdAddr: 0x12000000, RdAddr: 0x12000000, WrAddr: 0x11000000, RdAddr: 0x11000000, RdAddr: 0x11000000, WrAddr: 0x12000000, RdAddr: 0x12000000, RdAddr: 0x12000000, WrAddr: 0x13000000, RdAddr: 0x12000000, RdAddr: 0x12000000, WrAddr: 0x11000000, RdAddr: 0x11000000, RdAddr: 0x11000000, WrAddr: 0x12000000, RdAddr: 0x12000000, RdAddr: 0x12000000, WrAddr: 0x13000000, RdAddr: 0x12000000, RdAddr: 0x12000000, WrAddr: 0x11000000, RdAddr: 0x11000000, RdAddr: 0x11000000, and WrAddr: 0x12000000. The addresses alternate between 0x12000000, 0x13000000, and 0x11000000.

Se ha modificado el código del Callback para que utilce sólo 3 buffers:

```
void *TrBufFrmbufWrCallback(void *data)
{
```



```

{
    Frmbuf_t *frmbuf = (Frmbuf_t *)data;
    XV_FrmbufWr_l2 *frmbufwr = frmbuf->FrmbufWrPtr;
    XV_FrmbufRd_l2 *frmbufrd = frmbuf->FrmbufRdPtr;

    u32 Status;
    xil_printf("WrAddr: 0x%x\r\n", frmbuf->FrmbufWrBufAddr);

    // increase the number of frames written
    global_cntwr++;

    // Update the RdBufNumber only if the number of writes has reached NFRAMES_PARKED
    if (global_cntwr==NFRAMES_PARKED){
        global_cntwr=0;
        RdBufNumber= WrBufNumber;
        frmbuf->FrmbufRdBufAddr = frmbuf->FrmbufWrBufAddr;
    }

    // Always update the WrBufNumber depending on where the Rd channel is parked
    switch (RdBufNumber){
        case 0: // WR: 1->2->1

            //WrBufNumber = (WrBufNumber==1)? 2: 1;
            if (WrBufNumber==1){
                WrBufNumber=2;
                frmbuf->FrmbufWrBufAddr = frmbuf->FrmbufBufBaseAddr + (FRAME_SIZE *2);
            }
            else{
                WrBufNumber=1;
                frmbuf->FrmbufWrBufAddr = frmbuf->FrmbufBufBaseAddr + (FRAME_SIZE *1);
            }
            break;

        case 1:
            //WrBufNumber = (WrBufNumber==2)? 0: 2;
            if (WrBufNumber==2){
                WrBufNumber=0;
                frmbuf->FrmbufWrBufAddr = frmbuf->FrmbufBufBaseAddr + (FRAME_SIZE *0);
            }
            else{
                WrBufNumber=2;
                frmbuf->FrmbufWrBufAddr = frmbuf->FrmbufBufBaseAddr + (FRAME_SIZE *2);
            }
            break;

        case 2:
            //WrBufNumber = (WrBufNumber==0)? 1: 0;
            if (WrBufNumber==0){
                WrBufNumber=1;
                frmbuf->FrmbufWrBufAddr = frmbuf->FrmbufBufBaseAddr + (FRAME_SIZE *1);
            }
            else{
                WrBufNumber=0;
                frmbuf->FrmbufWrBufAddr = frmbuf->FrmbufBufBaseAddr + (FRAME_SIZE *0);
            }
            break;

        default:
            WrBufNumber = 0;
            frmbuf->FrmbufWrBufAddr = frmbuf->FrmbufBufBaseAddr + (FRAME_SIZE *0);
    }

    // Set the buffer address to write the next frame
    Status = XVFrmbufWr_SetBufferAddr(frmbufwr, frmbuf->FrmbufWrBufAddr);

    // Set the buffer address to read the next frame
    Status = XVFrmbufRd_SetBufferAddr(frmbufrd, frmbuf->FrmbufRdBufAddr);

}

```

Resultado:

— □ ×

1